

Take-Home Assignment

Course:	ENGR 5785G: Real-time Data Analytics for IoT
Topic:	Time Series Analysis: Traffic Prediction and Anomaly Detection
Due:	As in Canvas
Weight:	5% of course grade
Format:	Jupyter Notebook (.ipynb) + PDF Report
Groups:	Individual submission only

Purpose

This assignment extends the in-class traffic prediction tutorial. Students will build a short-term traffic forecasting workflow, compare classical and machine-learning methods, and identify abnormal traffic conditions from forecast residuals.

Assignment at a Glance

Dataset	Public PeMSD7 traffic-speed benchmark, or another instructor-approved traffic time-series dataset.
Core tasks	Exploratory analysis, preprocessing, ARIMA/SARIMA or exponential smoothing, ML with lag features, anomaly detection.
Expected duration	Approximately 4-6 hours outside class.
Submission format	One Jupyter notebook or Colab notebook, one short PDF report, and generated figures.
Assessment weight	100 marks.
Recommended tools	Python, pandas, NumPy, matplotlib, scikit-learn, statsmodels.

Learning Outcomes

- Load and prepare real traffic time-series data for forecasting.
- Construct lag, rolling-window, and calendar-based features.
- Apply at least one classical time-series forecasting model.
- Apply at least one machine-learning model for short-term traffic prediction.
- Evaluate forecasting models using time-aware validation and appropriate error metrics.
- Detect abnormal traffic conditions using model residuals or statistical thresholds.
- Communicate results clearly through plots, tables, and concise interpretation.

Dataset

Use the PeMSD7 traffic-speed benchmark dataset used in the in-class tutorial. The dataset contains traffic speed measurements from freeway sensors aggregated at 5-minute intervals. Students may use one sensor for a basic solution or multiple sensors for an extended solution.

Dataset access

Recommended source: PeMSD7_V_228.csv from the public STGCN traffic forecasting benchmark archive. If the download link changes, students may use METR-LA, PEMS-BAY, PeMSD4, PeMSD7, or PeMSD8 from a public traffic-forecasting benchmark repository, provided the report clearly documents the source.

```
# Recommended dataset-loading starting point
```

```
import pandas as pd
```

```
url = "https://github.com/VeritasYin/STGCN_IJCAI-18/raw/master/dataset/PeMSD7_V_228.csv"
```

```
df = pd.read_csv(url, header=None)
```

```
print(df.shape)
```

```
df.head()
```

Tasks:

Task 1: Data Loading and Initial Exploration (15 marks)

- Load the dataset and select one traffic sensor column as the target series.
- Create a datetime index assuming 5-minute intervals. You may start from any reasonable reference date if the dataset does not include timestamps.
- Plot the raw speed time series for at least three days.
- Compute descriptive statistics: mean, median, standard deviation, minimum, maximum, missing-value count.
- Discuss whether the series shows daily pattern, peaks, low-speed periods, or unusual drops.

```
# Example timestamp construction
```

```
sensor_id = 0
```

```
series = df.iloc[:, sensor_id].copy()
```

```
series.index = pd.date_range(start="2012-05-01", periods=len(series), freq="5min")
```

```
series.name = "speed"
```

```
series.plot(title="Traffic speed time series")
```

Task 2: Preprocessing and Feature Engineering (15 marks)

- Handle missing or invalid values using an appropriate method, such as interpolation or forward filling.
- Create lag features for at least 1, 2, 3, 6, and 12 previous intervals.
- Create rolling mean and rolling standard deviation features using at least two window sizes.
- Create calendar features such as hour of day, day of week, and weekend indicator.
- Define a clear forecasting target, such as speed 15 minutes ahead or 30 minutes ahead.

```
# Example feature engineering
```

```
import numpy as np
```

```
data = pd.DataFrame({"speed": series})
```

```

data["speed"] = data["speed"].interpolate().ffill().bfill()

for lag in [1, 2, 3, 6, 12]:
    data[f"lag_{lag}"] = data["speed"].shift(lag)

for window in [6, 12, 24]:
    data[f"roll_mean_{window}"] = data["speed"].shift(1).rolling(window).mean()
    data[f"roll_std_{window}"] = data["speed"].shift(1).rolling(window).std()

data["hour"] = data.index.hour
data["dayofweek"] = data.index.dayofweek
data["is_weekend"] = (data.index.dayofweek >= 5).astype(int)

horizon = 3 # 3 x 5 minutes = 15 minutes ahead
data["target"] = data["speed"].shift(-horizon)
data = data.dropna()

```

Task 3: Classical Forecasting Model (20 marks)

- Fit one classical forecasting model: ARIMA, SARIMA, or exponential smoothing.
- Use only the training portion of the data for model fitting.
- Forecast the test period and compare predicted speed with actual speed.
- Report MAE and RMSE.
- Briefly explain the selected model order or smoothing configuration.

```

# Example ARIMA model on a smaller subset for speed
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

split = int(len(series) * 0.8)
train_y = series.iloc[:split].asfreq("5min").interpolate()
test_y = series.iloc[split:split+288].asfreq("5min").interpolate() # one day test example

model = ARIMA(train_y.iloc[-2000:], order=(2, 1, 2))
fit = model.fit()
forecast = fit.forecast(steps=len(test_y))

mae = mean_absolute_error(test_y, forecast)
rmse = np.sqrt(mean_squared_error(test_y, forecast))
print({"ARIMA_MAE": mae, "ARIMA_RMSE": rmse})

```

Task 4: Machine-Learning Forecasting Model (20 marks)

- Train at least one machine-learning model using the engineered features. Recommended models: linear regression, random forest, gradient boosting, or XGBoost.
- Use a chronological train/test split. Do not randomly shuffle the time series.
- Compare the ML model against a naive persistence baseline.
- Report MAE, RMSE, and MAPE if appropriate.
- Plot actual vs predicted values for at least one day of the test set.

```

# Example ML model
from sklearn.ensemble import RandomForestRegressor

```

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

feature_cols = [c for c in data.columns if c not in ["target"]]
X = data[feature_cols]
y = data["target"]

split = int(len(data) * 0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

baseline_pred = X_test["speed"] # persistence baseline
rf = RandomForestRegressor(n_estimators=100, max_depth=12, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

def metrics(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred))
    }

print("Persistence:", metrics(y_test, baseline_pred))
print("Random Forest:", metrics(y_test, rf_pred))

```

Task 5: Residual-Based Anomaly Detection (15 marks)

- Compute residuals between actual and predicted speed for the selected best model.
- Define an anomaly threshold using either residual z-score, percentile threshold, or median absolute deviation.
- Identify at least 10 anomalous timestamps or all detected anomalies if fewer than 10 occur.
- Plot the speed series and highlight detected anomalies.
- Interpret whether anomalies likely represent congestion, sensor noise, abnormal drops, or model failure.

```

# Example residual anomaly detection
import matplotlib.pyplot as plt

results = pd.DataFrame(index=y_test.index)
results["actual"] = y_test
results["predicted"] = rf_pred
results["residual"] = results["actual"] - results["predicted"]

threshold = results["residual"].abs().quantile(0.99)
results["anomaly"] = results["residual"].abs() > threshold
print(results[results["anomaly"]].head(10))

ax = results.iloc[:288][["actual", "predicted"]].plot(figsize=(10, 4), title="Actual vs predicted speed")
anom = results.iloc[:288]
anom = anom[anom["anomaly"]]
ax.scatter(anom.index, anom["actual"], marker="x", label="Detected anomaly")
ax.legend()

```

```
plt.show()
```

Task 6 - Short Report and Interpretation (15 marks)

- Write a concise report of 2-4 pages, excluding figures and code appendix.
- Include the dataset description, preprocessing decisions, selected models, evaluation table, plots, anomaly results, and interpretation.
- Discuss which model performed best and why.
- Mention at least two limitations of the experiment.
- Suggest one improvement for a real-time deployment scenario.

Submission Checklist

- Notebook file: .ipynb or shareable Colab link with all code executed.
- Short report: PDF format, 2-4 pages excluding appendix.
- At least three figures: raw time series, actual-vs-predicted forecast, anomaly plot.
- At least one model comparison table.
- Brief written explanation of preprocessing, model choice, and results.
- All external sources and datasets cited in the report.

Marking Rubric

Component	Marks	Assessment criteria
Data loading and exploration	15	Correct loading, timestamp construction, descriptive statistics, clear initial visualization.
Preprocessing and features	15	Appropriate missing-value handling, lag features, rolling features, calendar features, and valid target definition.
Classical forecasting model	20	Correct ARIMA/SARIMA/exponential-smoothing implementation, chronological split, metrics, and interpretation.
Machine-learning model	20	Proper ML feature matrix, no temporal leakage, baseline comparison, metrics, and prediction plot.
Anomaly detection	15	Reasonable residual/threshold method, anomaly plot, timestamp table, and interpretation.
Report quality	15	Clear writing, organized figures/tables, discussion of limitations, and reproducible submission.

Report Structure

1. Introduction: forecasting objective and selected dataset.
2. Data and preprocessing: missing values, feature engineering, target horizon.
3. Methods: baseline, classical model, ML model, anomaly method.
4. Results: model-comparison table and plots.
5. Discussion: best model, anomalies, limitations, and real-time considerations.
6. References.

Optional Advanced Extension (Bonus / Discussion)

- Train on multiple sensors instead of one sensor and compare performance across sensors.
- Use a longer forecast horizon, such as 30 or 60 minutes ahead.
- Add a gradient-boosting model or neural-network model.
- Use the PeMSD7 distance matrix to discuss spatial relationships among sensors.
- Compare anomaly detection using residual thresholds versus Isolation Forest.

Academic Integrity and Reproducibility

Students may use online documentation, Python libraries, and AI-assisted coding tools, but submitted work must be understood, reproducible, and written in the student's own words. Any external code or dataset source must be cited. The notebook must run from top to bottom without hidden manual steps, except for documented package installation or dataset download steps.

References and Dataset Notes

- Caltrans Performance Measurement System (PeMS): <https://dot.ca.gov/programs/traffic-operations/mpr/pems-source>
- STGCN IJCAI-18 public benchmark repository: https://github.com/VeritasYin/STGCN_IJCAI-18
- Keras traffic forecasting example using PeMSD7: https://keras.io/examples/timeseries/timeseries_traffic_forecasting/
- statsmodels ARIMA documentation: <https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html>
- scikit-learn RandomForestRegressor documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>